PPOrphIt: PPO Robot Manipulator Morphology Design

Steve Gillet
University of Colorado Boulder Robotics
Email: steve.gillet@colorado.edu

Jay Vakil
Email: Jay.Vakil@colorado.edu

***Abstract*—This paper aims to address the cost and overactuation of robotics used in manufacturing by leveraging reinforcement learning (RL) to optimize robot morphologies to specific use cases. RL is the right tool for this job because it can optimize over the large search space of possible robot morphologies and effectively explore large varieties of embodiments. The core methodology involves using a PPO model to iterate on the different morphologies and MuJoCo to evaluate their effectiveness in simulation. The evaluation signals will be success, path and energy efficiency, simplicity and manipulability index and these will be used to show the effectiveness of the PPO model at creating morphologies that are faster, more effective, and more efficient for particular tasks.**

***Index Terms*—reinforcement learning, robot morphology, manipulator design**

I. Introduction and Motivation

High costs, lack of tailorability, and lack of retrofitting are some of the most cited problems for the adoption of the robotics in industrial settings [1]. The most commonly used robotic manipulator in manufacturing is FANUC series of 6-DOF manipulators which cost from $17,500 to $400,000 depending on their capacity and customizations [2] [3]. Meanwhile [4] and [5] show that most manipulators are overactuated, less cost-efficient, and less effective for most tasks they are used in. We want to address this problem by creating an automation pipeline where you feed in a use case and you get out a robot specifically optimized for that use case. By matching the robot to the task we can ensure that each robot can perform its task effectively and only the parts needed for that task are used, reducing cost and increasing efficiency. Here we describe the process by which robot manipulator morphologies are optimized to particular tasks using a PPO agent.

We chose reinforcement learning because morphology design is an optimization problem over a complex, high-dimensional search space and RL can explore and learn optimal designs through trial-and-error in simulation [6]. We chose PPO because it is robust and on-policy and excels in continuous action spaces which is ideal for morphology design where there are continuous parameters like link lengths [7]. This is more effective than imitation learning because imitation learning can only use morphologies that already exist severely limiting sample space and innovation [8]. Supervised learning suffers from a very similar issue where labelled data would have to be provided which would be infeasible and there is no mechanism for exploration [9].

The core method is to use stable_baselines3 PPO implementation to iterate on the design parameters (number of links, joint types (X, Y, Z hinge, and Z actuation), and link lengths), use MuJoCo to simulate that iteration on a simple task using RRTConnect and damped least squares (DLS) inverse kinematics (IK) for the path planning, feed the results (success, efficiency, and manipulability index) back into the PPO model. We chose RRTConnect because it is simple and has been shown to be faster than other methods [10] which enables faster and more robust evaluation steps. We also chose DLS IK for a similar reason, it has been shown to be robust especially in situations where the target position is out of reach [11] which is inevitable in a situation where an agent is exploring morphologies.

The evaluation signals will be successful path planning, path length, energy cost, and manipulability index. The manipulability index is a measure of a manipulators ability to arbitrarily change the position and orientation of the end effector at a particular point, borrowed from [12]. The hope is that the better morphologies will be able to do the tasks more successfully, efficiently, and quickly while still maintaing some robustness and then we can vary the tasks slightly to find better morphologies for particular tasks. The tasks will be kept simple but varied in ways to test different types of movements like moving near the base, moving from near to far, different elevations, more linear movements. You can imagine that a lot of pick and place tasks are mostly linear and so might largely be done by linear actuators more efficiently.

The research questions we seek to answer: Can PPO be used to generate more efficient morphologies? What are the best evaluation signals to use? How does varying evaluation signals yield different results (does optimizing for path shortness yield more simple manipulators while optimizing for manipulability measure yield more complex manipulators?)?

II. Background and Related Work

Most of the research in this area codesigns morphology and control of modular, atomic robots like [13] which uses PPO, [14] which uses TD3, [15], and [16] which uses PPO and A3C for configuration and control of a reconfigurable, modular robot. More pertinent [17] uses DDQN to co-optimize morphology and control for modular robotic manipulators and

[18] which uses soft actor critic for morphology and control of quadraped robots to avoid evalutating performance in sim or real to save time. Our method will focus on manipulators with simple controllers to allow us to focus on morphology design. The method will use standard 6-DOF FANUC and 7-DOF Franka Emika Panda morphology as a baseline and contrast with PPO generated morphologies. PPO has been used effectively in some of the relevant research and allows for searching over the continuous space of manipulator parameters instead of the discrete space of different module types in the atomic, reconfigurable manipulator optimization. In the future, we would also like to try other RL approaches like Dreamer. Dreamer uses a 'world model' to predict outcomes and plan ahead which improves sample efficiency which might be even more effective by reducing computation needs [19].

III. Methods

The idea is to use PPO to iteratively optimize robot manipulator morphology parameters like number of joints, joint types (hinge X/Y/Z or slide in Z), and link lengths through simulation in MuJoCo. The agent will propose designs, evaluate them on tasks using IK and RRTConnect path planning, and update the policy based on rewards for task success, efficiency, and manipulability.

The algorithmic structure will follow a standard RL loop: At each episode the PPO agent samples a morphology configuration from its policy (parameterized as a Gaussian distribution over continuous actions which are mapped to continuous link lengths and discrete joint types and numbers using scaling and rounding). The configuration will be instantiated in a dynamically generated MuJoCo XML model, simulated for a pick-and-place task (ie. moving from start to goal positions with varying elevations, distances, and obstacles), and controlled using damped least-squares IK for joint angles and OMPL's RRTConnect for path planning in joint space. Rewards are then computed and the policy is updated.

The object is to maximize expected cumulative rewards, defined as $r = w_1 \cdot s - w_2 \cdot l - w_3 \cdot n + w_4 \cdot m - w_5 \cdot e$, where s is task success (100 if a path is able to be made from start to goal, 30 otherwise), l is path length, n is number of links, m is the manipulability index (joint Jacobian determinant [12]), and e is energy cost measured in mechanical power (torque x velocity) and w_i are the tunable weights. PPO minimizes the clipped surrogate loss:

$$\mathcal{L}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \max(1-\epsilon, \min(1+\epsilon, r_t(\theta)))\hat{A}_t \right) \right] + S[\pi_\theta](s_t) - \beta \cdot \mathbb{E}_t[(\hat{V}_\theta(s_t) - V_t^{\text{targ}})^2] \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is the advantage estimated via Generalized Advantage Estimation (GAE) [20], $\epsilon = 0.2$ bounds the ratio for stability, S is an entropy bonus, and β balances the value loss [7].

Architectures use multi-layer perceptrons from Stable Baselines3. The policy network has 3 hidden layers of 128 units each with tanh activation functions for actor and critic.

Update rules involve collecting trajectories over N episodes, computing advantages, and performing K epochs of minibatch SGD on the PPO loss with a clip ratio of 0.2.

The baseline will involved standard 6-DOF and 7-DOF manipulators to contrast with RL's exploration. As an ablation we will test removing the manipulability term from the reward function to assess the impact on design dexterity versus simplicity.

IV. Experimental Setup

We are going to evaluate our approach using simulated pick-and-place type tasks in MuJoCo physics engine sourced from the official MuJoCo library (version 3.3.7). Tasks will include:

1) Basic pick-and-place: moving the end-effector from an average height, average distance start position (e.g., [-0.5, 1.0, 1.0]) to a similar height, similar distance goal position (e.g., [0.6, 0.75, 1.0]) with minimal obstacles
2) Shelf pick-and-place: placing the start and goal in mostly vertical orientations with an obstacle in between (e.g., [0.25, 0.0, 0.3] to [0.26, 0.1, 0.8]).
3) Multiple tasks: placing multiple start ang goal positions for similar but opposite motions (e.g., [0.25, 0.25, 0.0] to [0.25, 0.25, 1.0], then [0.24, 0.26, 1.1] to [0.25, 0.23, 0.1]).
4) Contrained environments: using similar tasks but in a constrained environment where movement is restricted such as a shipping container or a corridor.

Observations are a dummy single-dimensional value since the policy is state-independent. Actions are continuous and mapped to discrete number of links, continuous link lengths, and discrete joint types via scaling and rounding. Rewards are computed per task as per the reward function ($r = w_1 \cdot s - w_2 \cdot l - w_3 \cdot n + w_4 \cdot m - w_5 \cdot e$) with the rewards averaged across tasks for multi-task problems. Episodes consist of a single step with termination after morphology evaluation.

For compute budget and reproducability, we will use a NVIDIA GeForce RTX 4060 GPU (or CPU fallback 11th Gen Intel(R) Core(TM) i9-11900KF @ 3.50GHz) via Stable Baselines3. The expected total environment frames will be approximately 1e6 (1000000 episodes $\times$ 1 steps with 1 morphology evaluation each step) which corresponds to 4-6 hours wall-clock time based on MuJoCo's 5000 steps/second benchmark on similar hardware. Experiments will use at least 3 random seeds for all training and evaluation with results logged via TensorBoard. We will report means with 95% confidence intervals ($\bar{x} \pm 1.96 \cdot \frac{s}{\sqrt{n}}$ where s is standard deviation and n is number of seeds).

Primary success metrics will be cumulative return (average reward over episodes).

V. Timeline and Milestones

W1 (Oct 27 - Nov 2): Steve sets up the simulation environment, including dynamic XML/model generation for morphologies and basic pick-and-place tasks with IK/RRTConnect control; success check: Simulate and validate at least 3 fixed morphologies completing tasks and being evaluated with some

kind of reward, owner Steve. W2 (Nov 3 - Nov 9): Steve implements PPO training on a simplified, mostly fixed morphology; success check: Able to generate simplified morphologies with PPO for basic reach task , owner Steve. W3 (Nov 10 - Nov 16): Integrate PPO with morphology optimization (e.g., sampling link lengths/joint types), with Jay assisting on enhanced environment and morphology generation; success check: PPO generates and evaluates 10 novel morphologies, improving average reward by 20% over random baselines on linear motion task, owners Steve (RL) and Jay (simulation integration). W4 (Nov 17 - Nov 23): Add enhanced tasks and reward structure; success check: implement and generate 5 tasks to compare against baseline and ablation showing simpler designs without manipulability term (e.g., avg. joints reduced by 1-2), owners Steve (RL experiments) and Jay (evaluations). W5 (Nov 24 - Nov 30): Finalize full evaluations on all tasks, generate plots (learning curves, morphology examples), and complete writing; success check: All metrics reported with means/CIs, paper draft ready with visuals of optimized vs. standard manipulators, all.

VI. Risks and Mitigations

Potential risks include sparse rewards, a lot of designs will probably fail completely leading to zero feedback, we hope to mitigate this using shaped rewards (like distance to goal bonuses). Another risk is training instability in the RL model resulting from high-dimensional action spaces, we hope to address this with hyperparameter sweeps via logging, regularization terms, and earling stopping based on validation rewards. Compute limits may constrain episode counts, we will handle this with initial testing, CPU fallback, trying more sample-efficient methods like Dreamer. Stochasticity may cause variance in results, we counter this with 3+ random seeds and confidence intervals. Lastly, dependency on team roles may risk delays in integration, we hope to mitigate this with weekly meetings and shared code repositories.

VII. Resources

We are using the following software libraries and simulators:

A. Simulators

MuJoCo physics engine version 3.3.7 and Isaac Lab for robot dynamics. Both are open source under Apache 2.0 license.

B. RL Libraries

Stable Baselines3 version 2.7.0 for PPO implementation which is licensed under MIT and DreamerV3 which is open source under Apache 2.0.

C. Optimization & Planning

OMPL version 1.7.0 open source under BSD license for motion planning (shocking).

D. Data Processing

NumPy version 2.2.6 Copyright (c) 2005-2024, NumPy Developers. Used for array operations and reward calculations. Matplotlib version 3.10.6 Matplotlib Development Team license. Used for plotting learning curves.

E. Logging

TensorBoard version 2.20.0 licensed under Apache 2.0. Used for experiement tracking and metrics visualizations.

F. Datasets

No external datasets are used, all data is custom generated via MuJoCo/Isaac Lab.

G. Starter Code

Custom Python scripts for morphology generation and RL integration, inspired by MuJoCo tutorials (available from https://mujoco.readthedocs.io/en/stable/overview.html) and Stable Baselines3 examples (github.com/DLR-RM/stable-baselines3). All modifications are original to this project.

VIII. Ethics and Safety

No human subjects or hardware involved, all experiments conducted in simulation. We want to help advance the robotics community by ensuring our work is reproducible and all code will be made available under MIT license upon project completion. No sensitive data is used, only randomly-generated, simulation data.

IX. Evaluation Plan

We will produce the folowing plots and tables to assess and visualize the results:

A. Learning Curves

Plots of average cumulative return vs. training frames for PPO, with shaded 95% confidence intervals over 3 seeds with separate curves for each task to show convergence.

B. Comparison Results

Tables of rewards for optimized versus baseline morphologies for all of the tasks.

X. Expected Results

We pose the following falsifiable hypotheses:

1) PPO generated morphologies will achieve at least 20% higher success rates and 15% shorter path lengths on pick-and-place tasks compared to standard 6-DOF designs due to exploration of task-specific configurations.
2) Incorporating the manipulability term in the reward function will yield designs with 25% greater dexterity (measured by manipulability index) but potentially higher complexity, testable via ablation.

Success is defined as hypothesis (1) holding true with optimized morphologies showing reduced overkill indicating RL's viability for cost-effective manipulator design. An informative negative result would occur if hypothesis (2) fails indicating reward shaping needs refinement or if RL underperforms baseline indicating probably limitations in high-dimensional morphology spaces.

XI. Results

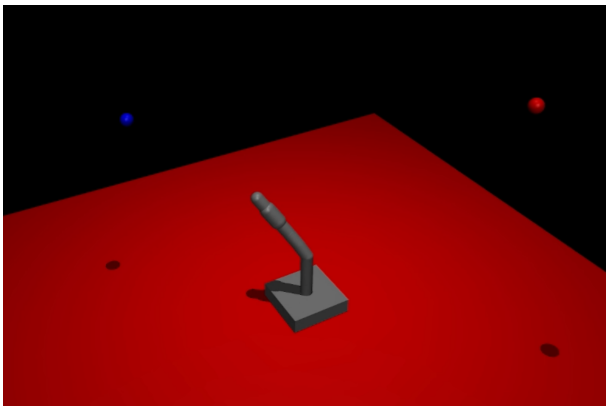

Fig. 1. Simple Pick and Place Task

We started with a simple environment with no obstacles and a simplified reward function (punishment for distance from start and goal points and for path length). In figure 1 you can see the blue and red dots representing the start and goal points and the first PPO optimized arm that we generated. Part of the learning and challenge of this method is shaping the reward function and you can see from the image that distance from start and goal was not punished enough resulting in an arm that could not actually reach the objectives. As you'll see in table I the baseline actually scores higher using the finalized reward function for this reason.

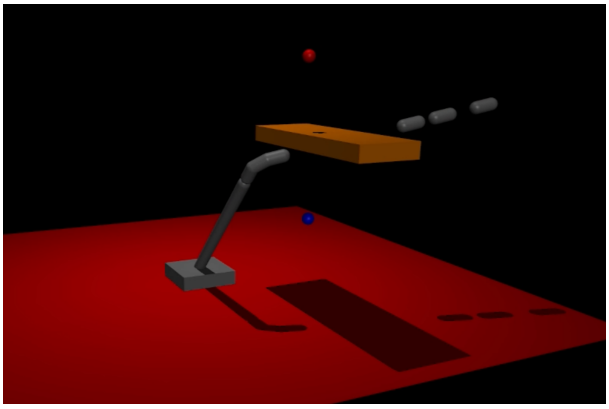

Fig. 2. Shelf Task

Then we increased the complexity of the task and the reward function by adding an obstacle and a bonus for manipulability index. As you can see in figure 2, the task now involves a more vertical movement but with an obstacle in the way the agent would have to learn to make an arm that can go around it efficiently. Unfortunately, the agent learned to go around it too efficiently, exploiting a weakness in the slide design where the slide joints did not have reasonable limits and only consisted of one component so that when they extended there was a physical gap in the arm that could be used to avoid obstacles. An important learning lesson at this point was that this agent is very good at exploiting weaknesses in the set up so that foundation has to be very strong to build upon.

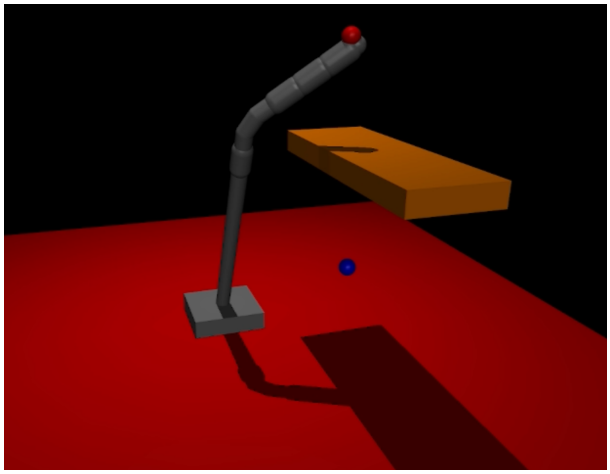

Fig. 3. Shelf Task Two Piece Slide Joints

In figure 3 you can see the fixed slide joint which is made of two pieces of the same length (slightly wider base piece and thinner slide piece) with a joint limit of the link length so that there is never a gap in the arm.

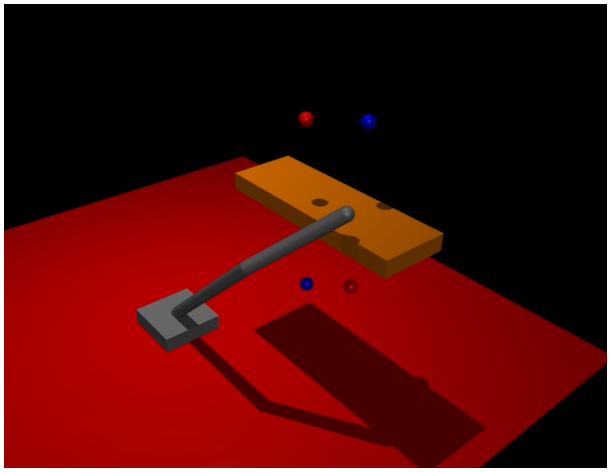

Fig. 4. Shelf with Multiple Tasks

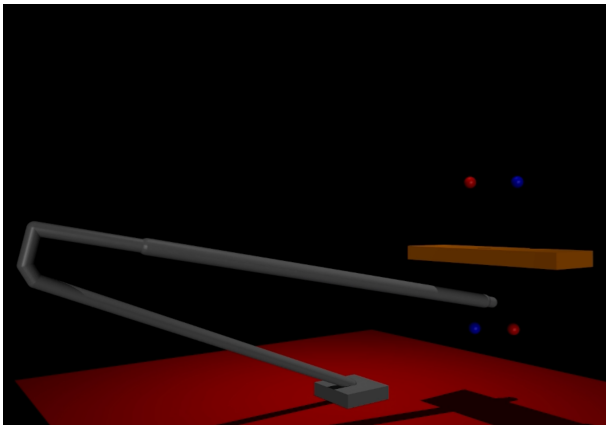

Fig. 5. Shelf with Multiple Tasks and Higher Manipulability Reward

Figures 4 and 5 show the experiments in adding tasks and tuning the reward function in the shelf task in an attempt to make arms that are more robust to small variations in the task and environment. An unexpected result of the manipulability index reward term is that slide joints contribute significantly to the manipulability index and so instead of the intended result of more complex, robust arms the result was more slide joints.

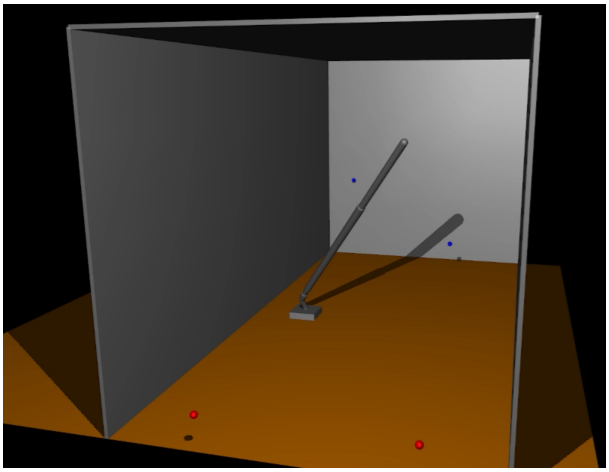

Fig. 6. Container Task (Constrained Environment)

For the constrained environment task, we surrounded the arm with wall and ceiling obstacles and put the start and goal points at either end as if the arm were unloading a truck. Figure 6 shows the environment and the resulting, optimized arm.

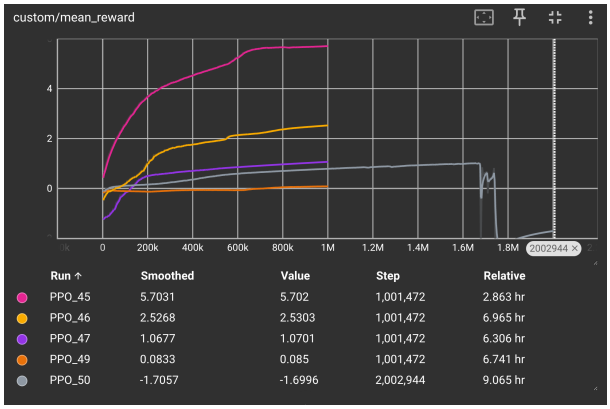

Fig. 7. Rewards per Episode Plot

Figure 7 shows the averaged, normalized rewards during the training of the various tasks. PPO_45 corresponds to the initial shelf task shown in figure 2, PPO_46 corresponds to the first multi shelf task (figure 4), PPO_47 to the second one with a higher manipulability score (figure 5), PPO_49 is the container task (figure 6), and PPO_50 shows the curve for another failed contrained task (figure 9) that will be discussed in the Conclusion section.

TABLE I
PERFORMANCE METRICS FOR PANDA AND OPTIMIZED ARM ACROSS TASKS

Task	Arm	Score	Success	Acc. Penalty	Manip. Bonus	Link Penalty	Path Penalty
Simple	Panda	86.7	100	-0.8	0.3	-12.5	-0.3
	Optimized	-48.86	100	-137	0.65	-12.5	-0.01
Shelf	Panda	85	100	-0.08	0.49	-12.5	-0.2
	Optimized	117	100	-0.15	30.37	-12.5	-0.09
Container	Panda	56.7	100	-31.2	1.2	-12.5	-0.8
	Optimized	115	100	-0.07	57	-12.5	-0.5

Table I shows the results of evaluating the task optimized arms versus a standard Franka Emika Panda baseline. Figure 8 shows the baseline morphology that was used to compare against. The basic joint types and dimensions of a standard Franka Emika Panda were used.

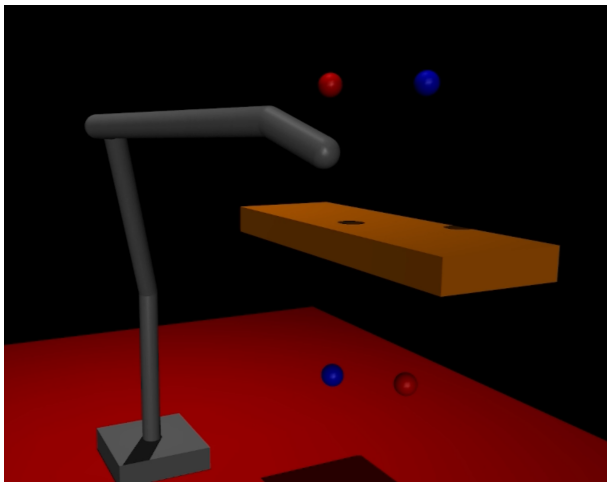

Fig. 8. Franka Emika Panda Baseline

The performance was better for the optimized arms than the baseline in each task except the simple task which was trained on an early iteration of the reward function. The optimized arms show shorter path lengths and higher manipulability with the accuracy results being mixed.

XII. Conclusion

The problem in using PPO to design a robot arm morphology is very much in the definition of the environment and the shaping of the reward. The optimized arms perform better in terms of the reward function they were optimized around. The arms generated and tested here got a bonus in manipulability measure by learning to use slide joints. However, using slide joints might not be meaningful in practice because the very directional manipulability that they add might not add anything practical.

Another issue with the optimized arms can be seen most notably in figure 6 where the arm has mostly small hinge joints at the base and then long slide joints that allow it to reach the start and goals and the path that it takes is very long tall arches as it goes from the start, up over its base, and back down the other side to the goals. This type of movement would require a lot of mechanical power in reality to hold the weight of the fully extended arm out against gravity using the motors in the small base joints.

This realization led to the final task environment and reward function that were not fully implemented in time for this paper. An energy cost term was added to the reward function and was calculated using MuJoCo's built-in inverse dynamics to get the mechanical power of the trajectories ($torque \cdot velocity$) and subtract that from the reward to punish higher energy using morphologies. Then we set up one last environment where the arm was mounted sideways on a wall and had start and goal positions from the floor onto a shelf as you can see in figure 9.

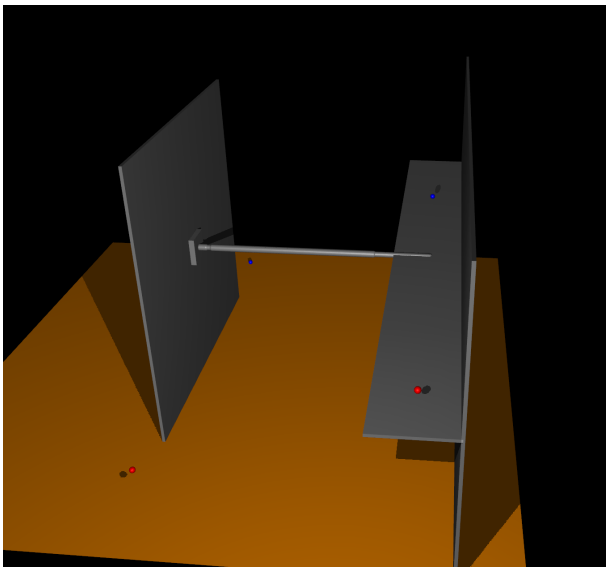

Fig. 9. Wall Mounted Task with Energy Cost Reward

If you look in figure 7 you can see the reward curve for this task (PPO_50) was quite low and hit some drop offs around 1.7 million episodes. Our hypothesis is that this task is a bit trickier than previous tasks and a lot more morphologies would fail because of the difficult start and goal placements and all of the obstacles and the models exploration led it into some unfortunate local minima as it went on.

Our hope for the future is to continue refining the environment and reward function. We want to remove the manipulability index and try other measures for robustness and complexity. Something I talked about in the proposal but did not implement here was introducing Guassian noise into the start and goal positions to provide some more robustness in the designs. We would also like to successfully implement the energy cost term and so might start that on an easier environment in the future.

XIII. Contributions and Roles

Steve Gillet is responsible for the RL implementation and initial simulation setup including PPO and baseline setups, reward design, ablations, and primary experiments on optimization and efficiency He owns the learning curves and rewards tables and RL-specific metrics in evaluations.

Jay Vakil handles the simulation environment including multi model generation in MuJoCo, REAL RL implementation, and any other evaluations. He owns the codebase maintenance and any other evaluation graphics used.

References

[1] McKinsey & Company, "Industrial robotics: Insights into the sector's future growth dynamics," McKinsey & Company, Tech. Rep., 2019. [Online]. Available: https://www.mckinsey.com/business-functions/operations/our-insights/industrial-robotics-insights-into-the-sectors-future-growth-dynamics

[2] Standard Bots, "Fanuc robot prices: Cost, models, and buying insights in 2025," 2025. [Online]. Available: https://standardbots.com/blog/fanuc-robot-price

[3] PatentPC, "Top robotics vendors by market share & installations," September 2025. [Online]. Available: https://patentpc.com/blog/top-robotics-vendors-by-market-share-installations

[4] M. Russo, L. Raimondi, X. Dong, and D. Axinte, "Task-oriented optimal dimensional synthesis of robotic manipulators with limited mobility," *Robotics and Computer-Integrated Manufacturing*, vol. 69, p. 102098, 2021.

[5] B. He, S. Wang, and Y. Liu, "Underactuated robotics: A review," *International Journal of Advanced Robotic Systems*, vol. 16, no. 4, pp. 1–14, 2019.

[6] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Machine Learning*, vol. 3, no. 1, pp. 9–44, 1988.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[8] J. M. D. Delgado and L. Oyedele, "Robotics in construction: A critical review of the reinforcement learning and imitation learning paradigms," *Advanced Engineering Informatics*, vol. 54, p. 101787, 2022.

[9] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.

[10] J. Kuffner and S. LaValle, "Rrt-connect: An efficient approach to single-query path planning," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, vol. 2, 2000, pp. 995–1001 vol.2.

[11] S. R. Buss, "Introduction to inverse kinematics with jacobian transpose, pseudoinverse and damped least squares methods," *IEEE Journal of Robotics and Automation*, vol. 17, no. 1-19, p. 16, 2004.

[12] T. Yoshikawa, "Manipulability of robotic mechanisms," *The International Journal of Robotics Research*, vol. 4, no. 2, pp. 3–9, 1985.
[13] J. Bhatia *et al.*, "Reinforcement learning for freeform robot design," *arXiv preprint arXiv:2310.05670*, 2023.
[14] B. Tjanaka *et al.*, "Co-optimization of morphology and behavior of modular robots via hierarchical deep reinforcement learning," in *Robotics: Science and Systems (RSS)*, 2023.
[15] A. Spielberg *et al.*, "Accelerated co-design of robots through morphological pretraining," *arXiv preprint arXiv:2502.10862*, 2025.
[16] M. Kalimuthu, A. A. Hayat, T. Pathmakumar, M. R. Elara, and K. L. Wood, "A deep reinforcement learning approach to optimal morphologies generation in reconfigurable tiling robots," *Mathematics*, vol. 11, no. 18, p. 3893, 2023.
[17] Z. Ding, H. Tang, H. Wan, C. Zhang, and R. Sun, "A modular robotic arm configuration design method based on double dqn with prioritized experience replay," *Symmetry*, vol. 16, no. 6, p. 714, 2024. [Online]. Available: https://www.mdpi.com/2073-8994/16/6/714
[18] K. S. Luck, H. B. Amor, R. Calandra, and J. Peters, "Data-efficient co-adaptation of morphology and behaviour with deep reinforcement learning," in *Conference on Robot Learning (CoRL)*, 2019.
[19] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Dream to control: Learning behaviors by latent imagination," *arXiv preprint arXiv:1912.01603*, 2019.
[20] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.